

Researching ways to mask activity on Snapchat through preexisting tools

Introduction

The police uses Snapchat to talk to suspected individuals, through fake/controlled accounts portraying to be underage children, using AI generated images. The goal of our project is to find a way to mask the activity of the police and extract the IP address of the suspects, so that they can then use it to find out the location of the suspect.

Masking is defined as:

- Make it appear as if messages sent on Snapchat are being sent on a mobile device, not a browser, as Snapchat notifies users from where messages are being sent through
- Images that are being sent need to appear as images that were taken through Snapchat, and not as images directly uploaded through a camera roll, as those appear different in Snapchat
- Making an account look relatively active (snap score not 0, aka not 0 messages sent in total, as it appears that the account has not been used yet)
- Not notifying the suspect of screenshots taken of the conversation, needed for collection of evidence

All research and testing is done on accounts that are either owned by us (the team that is working on the project) or the stakeholders (police Rotterdam, Fontys coaches). No non-consenting parties are used in testing/experimenting. These efforts are not to be used in any scope outside the project, as that goes against the policy of Snapchat. All evidence found is to be kept between the Stakeholders and us.

Context

In this document, I analyze the use of Snapenhance to characterize its features for potential investigative support, before considering custom solutions. The purpose is to understand existing tools' operations and identify limitations.

Snapenhance is a popular tool for enhancing Snapchat functionality. An AI summary of its use is:

"For Android users, SnapEnhance is an Xposed module designed to enhance the Snapchat experience. It requires root access and the Xposed framework to function, offering features like preventing screenshot notifications, disabling read receipts, downloading messages and stories, hiding presence, and customizing the user interface. This version is actively developed, though it may contain bugs and crashes."

Objective

Go through the setup of Snapenhance across multiple devices to establish a more "exploit-able" direction the project will take, (emulated, physical phone, PC).

Scope

Controlled testing on personally owned devices or consenting test accounts.

Process and Attempts

Below, I describe the three methods attempted using Snapenhance, including setup, observations, and limitations.

1. Emulated Android Device

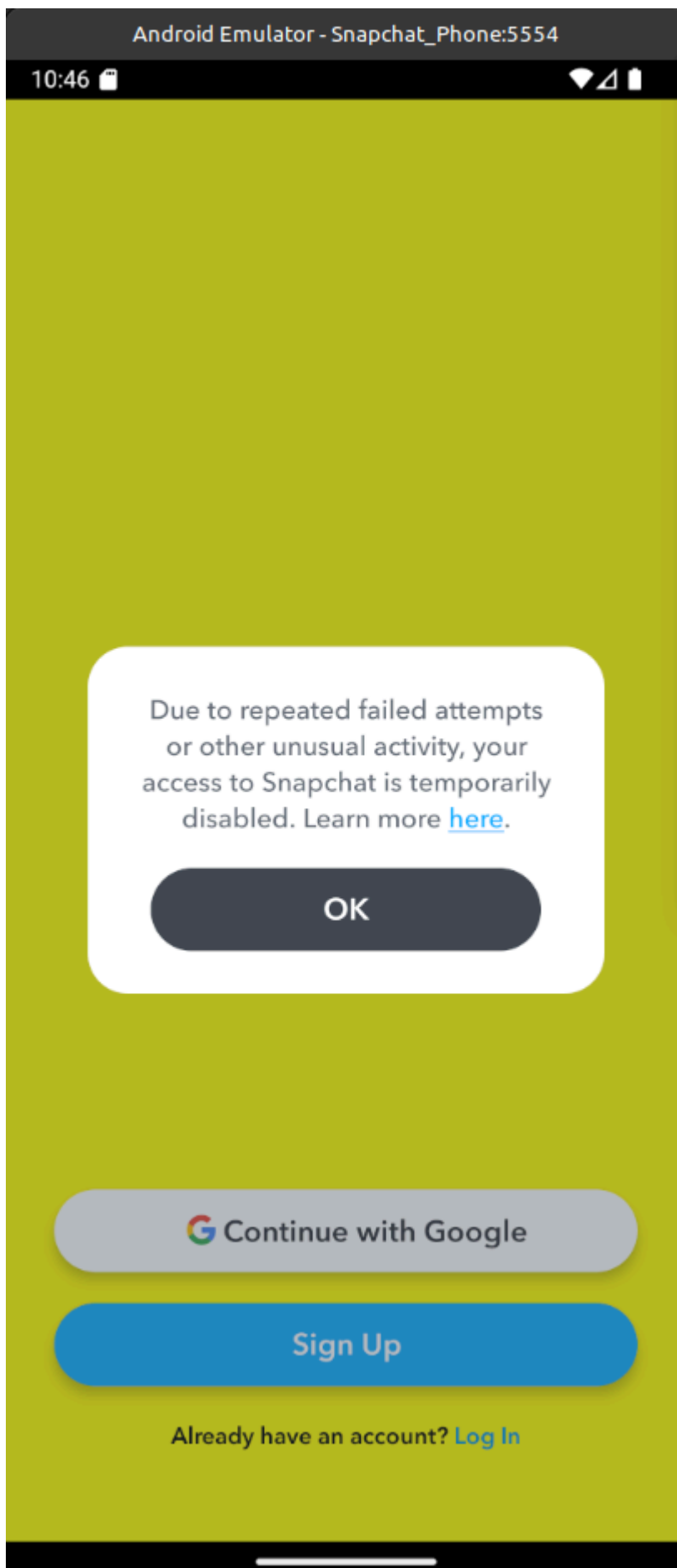
Environment: Android Studio emulator (emulated hardware mimicking a mobile phone), rooted with Magisk. OS: Android 14. App Version: Snapchat 12.33.1.19 (recommended to avoid bans, downloaded as APK).

Test Steps:

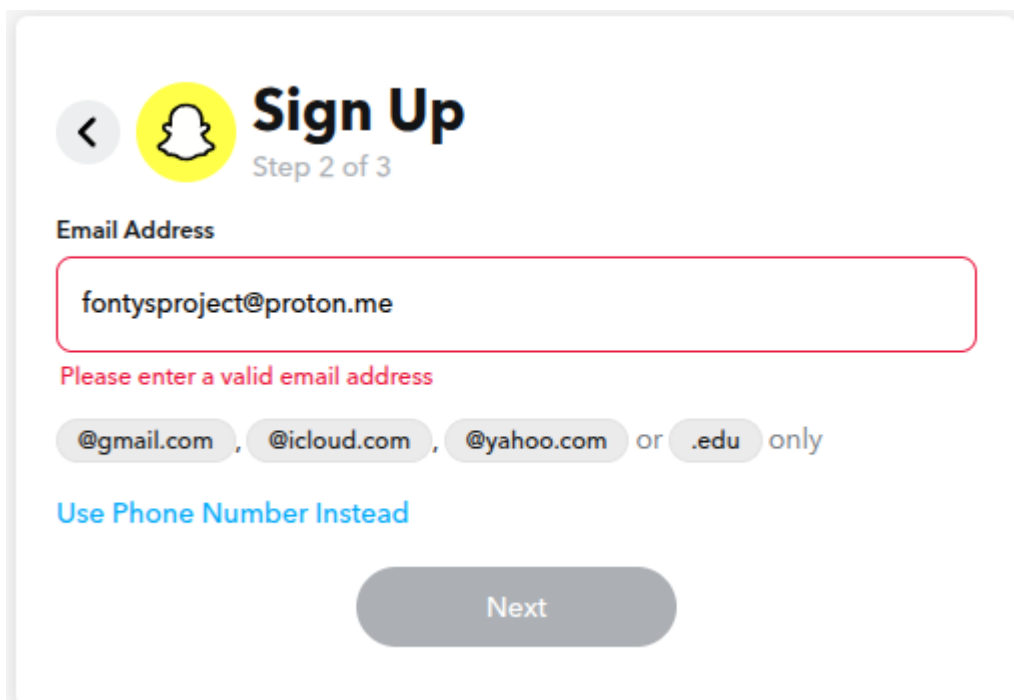
1. Set up a premade Docker-containerized VM rooted with Magisk.
2. Attempted to host and run Snapchat APK.
3. Switched to custom Android Studio VM for better hardware emulation.
4. Attempted to download official Snapchat from Play Store or via APK.
5. Tried account creation and login.

Observed Outcome:

- Premade VM incompatible with APK hosting due to hardware takeover from PC.
- Custom VM identified as virtual by Snapchat, preventing Play Store downloads.
- APK installation allowed, but login failed due to detection of suspicious activity (e.g., outdated app, modified environment, hardware flags).
- Account creation required email with phone registration to prevent spam (Gmail, Yahoo, iCloud, .edu accounts); test accounts risked bans.



Screenshot from 2025-09-22 22-46-08.png



Screenshot from 2025-09-22 22-40-13.png

This environment seemed like the most attractive one as it would make the process very easily reproducible on other devices.

The emulated hardware detection however makes this a tricky solution.

2. Physical Android Device

Environment: Physical Android phone (model: One Plus 8 Pro), rooted with Magisk. OS: Oxygen OS 13. App Version: Older Snapchat APK (pre-12.33.1.19).

Test Steps:

1. Root device with Magisk and install required modules (e.g., LSPosed).
2. Install older Snapchat APK.
3. Attempt login and feature testing.
4. Hide Magisk and retry.

Observed Outcome:

- APK required upgrade to newer version.
- Official Snapchat detected compromised device, even after hiding root.
- No successful login or feature access.

The idea behind this solution was that the phone could be connected to a PC and use it from there, and finding a solution to the log exporting etc. at a later

time, once it was verified that it was actually working.

It seems like a very big hassle to get it functioning though, and at this point I still had another option to try out, the web version of Snapchat.

Snapchat Web Version

Environment: Chrome browser (version 138.0.7204.183) on Linux Mint.

Extensions: Tampermonkey.

Test Steps:

1. Install Tampermonkey extension.
2. Load Snapenhance web userscript.
3. Test on example.com for setup verification.
4. Apply to Snapchat Web and test image download.

Observed Outcome:

- Setup straightforward, script loaded successfully after minor adjustments.
- Enabled features like image downloads, which are restricted in standard web client.
- No detection issues observed in this controlled test, though web client may reveal browser origin.

Attached separately I have a video that demonstrates how Snapenhanced allows for downloading images from Snapchat Web, which is normally not allowed. Moreover, a backup link has been posted of the video on YouTube: https://youtu.be/yw_012c2VBM.

Conclusion and Next Steps

The three tests characterize Snapenhance's feasibility across environments: emulation and physical devices face significant platform detection hurdles, while web setup is simpler but limited in mimicking mobile behaviors. Overall, these observations highlight the need for lawful alternatives rather than custom evasion.

Next Steps:

1. Get approval from the Stakeholder to get in contact with Snapchat to exclude a simpler "legal" solution.
2. Expand testing: Continue researching Snapchat Web version with Snapenhance to understand the way it works/what further it masks.
3. Compare checks done between mobile and web based platform (API, other back-end related items).
4. Review research document with an expert.

Sources

hub.docker.com/r/shmayro/dockerify-android

Mahendral. (2025, September 10). [GUIDE] Easiest way to root/magisk Oneplus 8 Pro [Online forum post]. XDA Forums.

<https://xdaforums.com/t/guide-easiest-way-to-root-magisk-oneplus-8-pro.4288757/>

Hyper NotePad. (n.d.). <https://hypernotepad.com/n/ec03baa34f4665df>